Krishna Kumar Trivedi Exp-4

```
# Upload kaggle.json first
from google.colab import files
files.upload()
```

Choose Files boston-hou…-dataset.csv

- **boston-housing-dataset.csv**(text/csv) - 39060 bytes, last modified: 3/3/2026 - 100% done

```
Saving boston-housing-dataset.csv to boston-housing-dataset.csv
{'boston-housing-dataset.csv':
b',CRIM,ZN,INDUS,CHAS,NOX,RM,AGE,DIS,RAD,TAX,PTRATIO,B,LSTAT,MEDV\n0,0.00632,18.0,2.31,0,0.538,6.575,65.2,4.09,1,296.0,15.3,
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
df = pd.read_csv("boston-housing-dataset.csv")
df.head()
```

	Unnamed: 0	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90	4.98	24.0
1	1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90	9.14	21.6
2	2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	34.7
3	3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	33.4
4	4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90	5.33	36.2

Next steps: Generate code with df New interactive sheet

```
df.isnull().sum()
```

	0
Unnamed: 0	0
CRIM	0
ZN	0
INDUS	0
CHAS	0
NOX	0
RM	0
AGE	0
DIS	0
RAD	0
TAX	0
PTRATIO	0
B	0
LSTAT	0
MEDV	0

dtype: int64

```
X = df.drop("MEDV", axis=1)
y = df["MEDV"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
model = LinearRegression()
model.fit(X_train_scaled, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

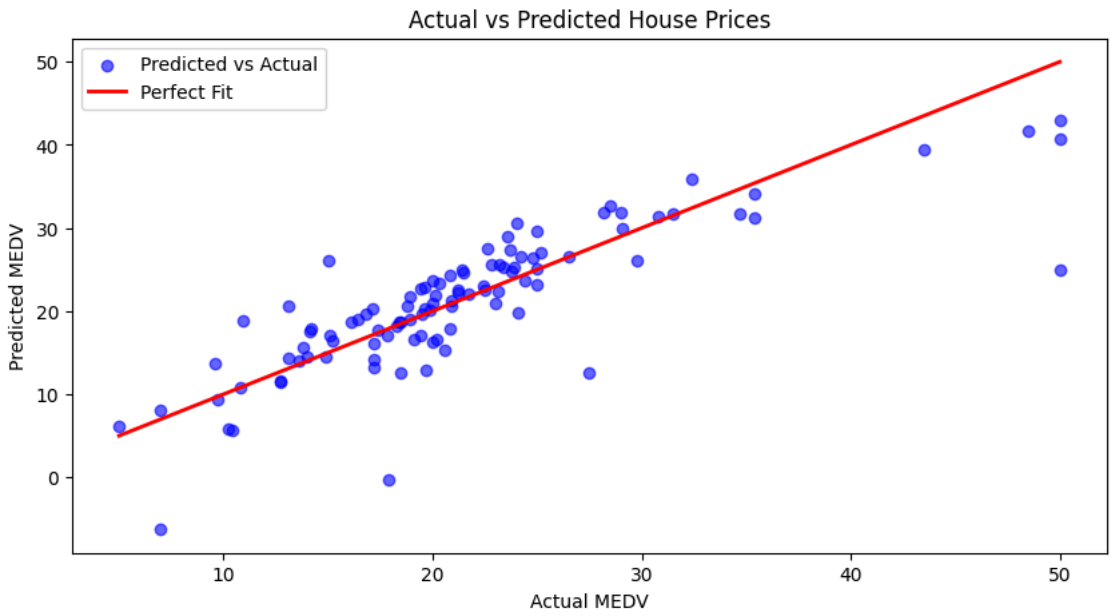
```
y_pred = model.predict(X_test_scaled)
```

```
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print("R2 Score:", r2)
print("Mean Squared Error:", mse)
print("Root Mean Squared Error:", rmse)
```

```
R2 Score: 0.6659408703343038
Mean Squared Error: 24.497819777630372
Root Mean Squared Error: 4.949527227688557
```

```
plt.figure(figsize=(10,5))
plt.scatter(y_test, y_pred, color='blue', alpha=0.6, label='Predicted vs Actual')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', linewidth=2, label='Perfect Fit')
plt.xlabel("Actual MEDV")
plt.ylabel("Predicted MEDV")
plt.title("Actual vs Predicted House Prices")
plt.legend()
plt.show()
```



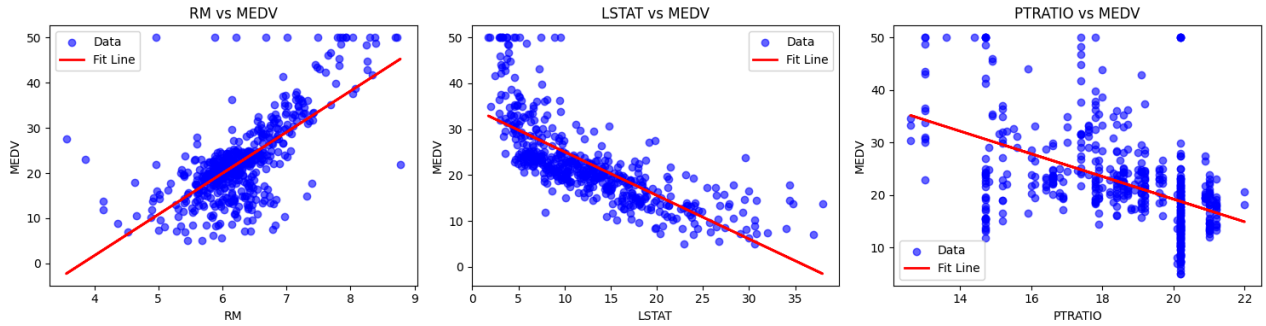

```
# Features to plot (3 important features)
features_to_plot = ["RM", "LSTAT", "PTRATIO"]

# Create a figure
plt.figure(figsize=(15, 4))

# Loop through each feature
for i, feature in enumerate(features_to_plot):
    plt.subplot(1, 3, i + 1)

    x = df[feature]
    y = df["MEDV"]

    # Scatter plot of the data
    plt.scatter(x, y, color='blue', alpha=0.6, label='Data')

    # Fit a linear regression line
    m, b = np.polyfit(x, y, 1)  # slope and intercept
```

```
    plt.plot(x, m * x + b, color='red', linewidth=2, label='Fit Line')

    # Labels and title
    plt.xlabel(feature)
    plt.ylabel("MEDV")
    plt.title(f"{feature} vs MEDV")
    plt.legend()

# Adjust layout and show plot
plt.tight_layout()
plt.show()
```